

Seeing What a Network Sees

From heatmaps to circuits, in an image model

Two heatmaps you have already seen

Chapter 6 - Grad-CAM

Where in the photo did the network look when it said “pomegranate”? A heatmap over the pixels.

Chapter 5 - LIME

Which patches of the husky photo pushed the answer to “wolf”? The snow. A heatmap over the pixels.

Both answer “**which part of the input** mattered”. Neither opens the network: they say nothing about *what* it computed from those pixels, or *how*.

Today we start with heatmaps - and find out how much they can be trusted - and then go inside: what does a single neuron of ResNet-18 want to see, and how is it built from the neurons before it?

Optional lecture. It needs only the introduction.

The model

ResNet-18, trained on ImageNet (1000 classes) - the same model as chapter 6's Grad-CAM, 11.7 million parameters. Four stages of convolutional blocks:

stage	channels	each position sees about ...
layer1	64	a small patch
layer2	128	a larger patch
layer3	256	most of an object part
layer4	512	most of the image

What it says about four photos from this course's own folders:

photo	ResNet-18's answer	runner-up
a pomegranate	pomegranate 99%	strawberry 0.4%
two puppies	Border terrier 27%	gorilla 25%
a sheep	bison 35%	brown bear 20%
a KAMAZ truck	jeep 67%	tractor 5%

Outline

Gradients as explanations

Can a heatmap be trusted?

Feature visualisation

From neurons to circuits

Gradients as explanations

Which pixels would change the answer most?

The idea: a saliency map

The network computes a score for the class, $f(x)$, from the pixels x . Ask the calculus question: if I nudge one pixel, how much does the score move?

$$\text{saliency}(\text{pixel } i) = \left| \frac{\partial f}{\partial x_i} \right|$$

One backward pass gives it for every pixel at once - it is backpropagation, stopped at the input instead of the weights.

By hand, for a linear model $f(x) = w_1x_1 + w_2x_2 + b$: the gradient is just (w_1, w_2) . The saliency map of a linear model is its coefficients - the glass-box reading from chapter 5. For a deep network the gradient changes from input to input, which is the whole point, and the whole difficulty.

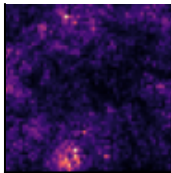
Simonyan, Vedaldi & Zisserman (2013).

Three heatmaps of the same decision

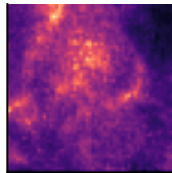
pomegranate (99%)



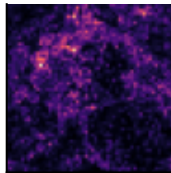
gradient



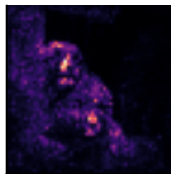
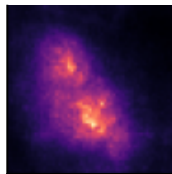
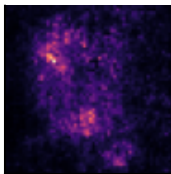
SmoothGrad



integrated gradients



Border terrier (27%)



Gradient: speckled - one pixel's gradient is very noisy. **SmoothGrad:** the gradient averaged over 16 noisy copies of the photo - smoother (Smilkov et al., 2017). **Integrated gradients:** the next slides.

The flaw in a plain gradient

Take $f(x_1, x_2) = \min(1, x_1 + x_2)$ and the input $(1, 1)$. Both inputs clearly matter: set both to 0 and the output falls from 1 to 0.

But at $(1, 1)$ the sum is 2, well past the cap of 1, so nudging either input changes nothing:

$$\frac{\partial f}{\partial x_1} = \frac{\partial f}{\partial x_2} = 0.$$

The gradient says neither input matters. The network is **saturated** - exactly the situation of a confident deep network, whose ReLUs and softmax flatten out once it is sure.

Integrated gradients, by hand

Instead of one gradient at the input, **average the gradient along the straight path** from a baseline x' (here $(0,0)$) to the input x , and multiply by how far each input moved:

$$IG_i = (x_i - x'_i) \int_0^1 \frac{\partial f}{\partial x_i} (x' + t(x - x')) dt$$

On the path, $x_1 + x_2 = 2t$. For $t < 0.5$ the cap is not hit and $\partial f / \partial x_1 = 1$; for $t > 0.5$ it is 0:

$$IG_1 = (1 - 0) \left(\int_0^{0.5} 1 dt + \int_{0.5}^1 0 dt \right) = 0.5, \quad IG_2 = 0.5.$$

$IG_1 + IG_2 = 1 = f(1, 1) - f(0, 0)$. That is not a coincidence: the attributions always add up to the change in the output (**completeness**). It is the same idea as Shapley values in chapter 5 - split the total change fairly between the inputs.

Completeness, checked on ResNet-18

The integral is approximated with a finite number of steps along the path. With more steps the sum of all 150,528 pixel attributions should approach $f(x) - f(\text{black image})$:

steps	sum of attributions	$f(x) - f(\text{black})$	off by
8	22.66	18.45	+23%
32	16.86	18.45	-9%
128	18.56	18.45	+0.6%
512	18.455	18.450	+0.03%

The pomegranate photo, score of the class “pomegranate”. The two-puppies photo behaves the same way (+0.5% at 512 steps).

The **baseline is a choice** with consequences. With a black baseline, a pixel that is already black has $x_i - x'_i = 0$ and gets **zero** attribution, however much the network relies on it. A different baseline gives a different map.

Can a heatmap be trusted?

Destroy the network and see if the explanation notices

The sanity check

An explanation of a model's decision should **depend on the model**. So:

1. compute the heatmap on the trained network;
2. replace the weights of the **top** layer with random numbers, and compute it again;
3. randomise the next layer down too, and again - until nothing trained is left.

If the heatmap stays the same while the network turns to noise, it was never showing what the network learned. It was showing something about the *photo* - usually its edges.

Adebayo et al. (2018), *Sanity Checks for Saliency Maps*.

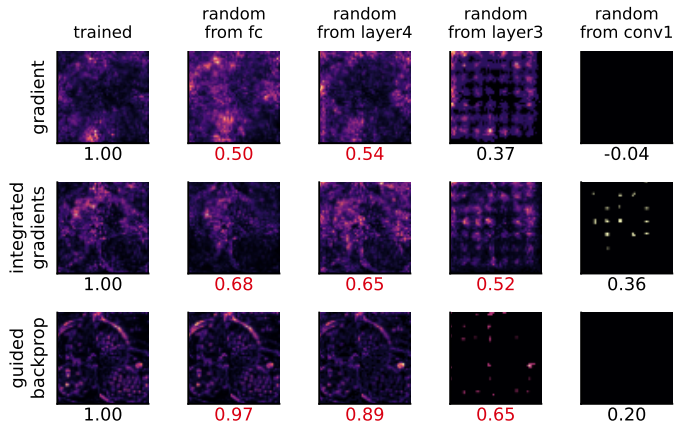
Predict first

Three methods: the plain gradient, integrated gradients, and **guided backprop** - a popular method that produces especially crisp, photo-like maps (it lets only positive gradients flow back through each ReLU).

Randomise only the **top** of the network - the classifier and the last stage, the part that makes the decision.

Which of the three maps will barely change?

The result



Pomegranate photo, network randomised from the top down (fc = the classifier). Under each map: rank correlation with the trained map (1 = same pixel ranking); red = still over 0.5.

What that means

- ▶ **Guided backprop** barely notices: with the classifier and then the whole last stage random, its map still matches the trained one at **0.97** and **0.89**. It was drawing the photo's edges, not the decision.
- ▶ The **plain gradient** changes at once (0.50), and is gone when nothing trained is left.
- ▶ **Integrated gradients** changes too, but keeps 0.36 with nothing trained left: it multiplies by the pixel values, so part of every IG map is the photo itself.

A heatmap can look sharp, plausible and object-shaped and still depend on **nothing the model learned**. Crispness is not evidence. The only test is the one from the introduction: **intervene** - here, on the weights - and see whether the explanation moves.

And even the heatmaps that pass answer only “*where* in the input”. The rest of this lecture asks the question heatmaps cannot: **what is the network detecting**, and how is it built?

Feature visualisation

Ask a neuron what it wants to see

The idea

Saliency fixes the image and asks about the pixels. Turn it around: fix a **neuron** (one channel of one layer) and **optimise the image** to make that channel fire as strongly as possible.

$$x^* = \arg \max_x \text{activation}_{\text{channel}}(x)$$

Gradient *ascent* on the pixels, with the network frozen. Whatever picture comes out is what that channel is looking for.

It is saliency run backwards. Saliency asks which pixels move the output; this asks which picture moves one *internal* unit - the first question in this lecture about the inside of the network rather than its input.

Erhan et al. (2009); Olah, Mordvintsev & Schubert (2017), *Feature Visualization*.

Naive optimisation gives noise

Optimise every pixel freely and the result is busy high-frequency texture - an **adversarial pattern** that excites the channel but is not a picture of anything.

Right: the same channel (layer3, 43) both ways.

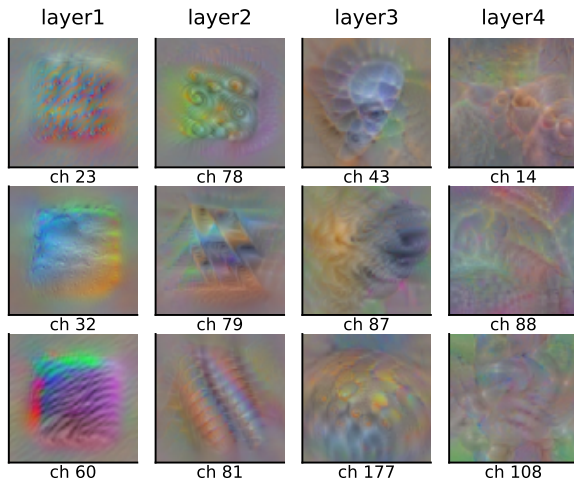


The standard fixes, used for every picture from here on:

- ▶ optimise a **smooth** image: its Fourier coefficients, with high frequencies scaled down;
- ▶ at every step, **jitter, rescale and rotate** the image slightly - the picture has to excite the channel from every nearby viewpoint, which noise cannot do.

These are **priors**, chosen by us. A visualisation shows what excites the channel *among smooth, shift-robust images* - not everything that excites it.

Four stages, twelve random channels



Three channels per stage, **picked at random** (seeded), not chosen for looking good.

layer1: colour and fine stripes. layer2: repeated textures - swirls, bands.

layer3: rounded shapes, scales. layer4: hard to name - with these priors most deep channels do not give a clean picture.

aggregated Going deeper, the pictures get bigger and more structured - stripes, then textures, then parts. The deepest ones are the hardest to read.

From neurons to circuits

A curve detector, and what it is built from

The curve detectors

The most studied neurons in any vision model are the **curve detectors** of InceptionV1: channels that fire on a curve of one orientation, not on straight lines and not on curves facing the other way. Their circuit was traced by hand, neuron by neuron.

The universality claim from the introduction predicts that other networks trained on images grow curve detectors too. ResNet-18 was never studied that way. Can we find one in it ourselves?

Cammarata et al. (2020), *Curve Detectors*; Olah et al. (2020), *Zoom In*.

The hunt

Show every channel of layer1 and layer2 two sets of synthetic images, 16 orientations each:

- ▶ a **quarter-circle arc** through the centre, facing each direction;
- ▶ a **straight line** through the centre, at each angle.

Call a channel **curve-like** if its best arc response is at least twice its best line response, it prefers some orientations clearly over others, and it responds strongly (top quarter of the stage).

stage	channels	curve-like	best one: arc vs line
layer1	64	1	channel 12: 5.27 vs 2.54
layer2	128	6	channel 23: 4.42 vs 1.15

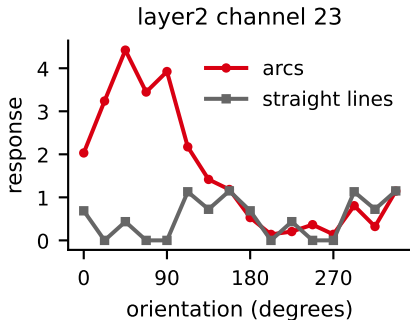
layer1's one candidate barely passes (2.1 times). layer2 has several clear ones.

The best candidate

preferred stimulus



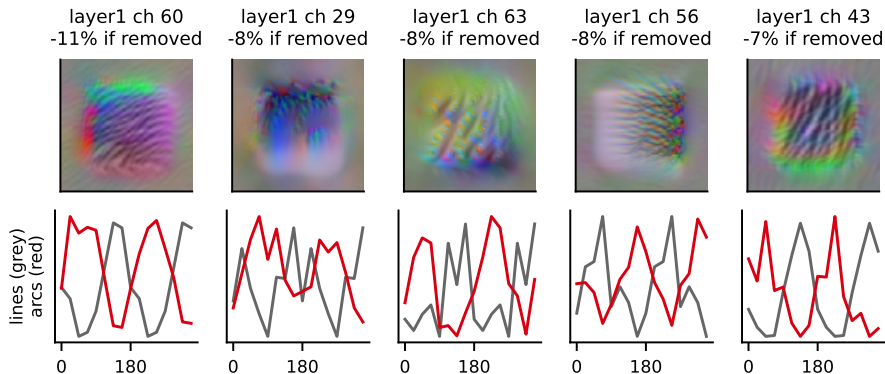
feature visualisation



layer2, channel 23. Its best arc gives **4.42**, its best straight line **1.15** - almost four times less. And it is tuned: arcs facing one way (around 45°) excite it, arcs facing the opposite way ($180\text{--}270^\circ$) barely do. Its feature visualisation is a stack of arcs, all bending the same way.

What is it built from?

Show the curve detector its preferred arc, then **switch off one layer1 channel at a time** and measure how much its response falls.



Top: the five layer1 channels it depends on most, with their feature visualisations. Bottom: their responses to straight lines (grey) and arcs (red), by orientation.

Edges lying along the curve

No single input carries it. Removing any one layer1 channel costs at most 11%; it takes 8 of the 64 to remove half the response.

But the inputs are not random. All top five are **edge detectors** - they swing strongly with the angle of a line. Which angles?

- ▶ The preferred arc faces 45° . Along it, the direction of the curve turns from 90° to 180° (its tangents).
- ▶ Four of the five top inputs prefer lines at **135° or 157.5°** - inside that range. The fifth prefers 67.5° , outside it.

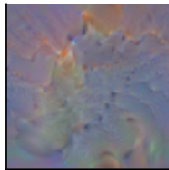
A curve assembled from small edges lying along it - the same recipe found by hand in InceptionV1, here spread over more inputs, with none worth more than 11%.

One neuron, several meanings

channel 507
spider web
garden spider
black and gold garden spider



channel 337
triceratops
pencil sharpener
lionfish



channel 254
basketball
triceratops
volleyball



Last-layer channels whose two strongest class votes (read straight off the final weights) fall on both sides of ImageNet's animal / object split: **107 of 512** pass.

The test is crude: channel 507 passes only because ImageNet files “spider web” as an object - it is plainly one concept, spiders. Channels 337 and 254 are genuinely mixed.

Some units mean one thing, others several. A network with more concepts than channels has to share channels between concepts - **superposition**, which comes back in the features lecture.

Recap

- ▶ **Saliency** = the gradient of the class score with respect to the pixels. Noisy; SmoothGrad averages the noise away.
- ▶ **Integrated gradients** averages the gradient along a path from a baseline, and its attributions add up to the change in the output - checked on ResNet-18.
- ▶ **Sanity checks**: guided backprop ignored a randomised top of the network (0.97); a crisp map is not evidence. Integrated gradients keeps part of the photo in every map.
- ▶ **Feature visualisation** asks a channel what it wants to see; the answers grow from edges to parts with depth, under priors we chose.
- ▶ **Circuits**: a curve detector in ResNet-18 (layer2, channel 23), built from edge detectors lying along the arc - but spread over many inputs, none worth more than 11%.

Heatmaps say *where*. Neurons and circuits say *what* and *how* - and every claim still has to survive an intervention.

Next core lecture: opening the box.

The same questions, on a language model - where you cannot look at a feature with your own eyes, but where the architecture makes something vision models do not: the whole computation adds up into one stream, exactly.